



Proximity Forest: an effective and scalable distance-based classifier for time series

Benjamin Lucas¹ · Ahmed Shifaz¹ · Charlotte Pelletier¹ ·
Lachlan O'Neill¹ · Nayyar Zaidi¹ · Bart Goethals¹ · François Petitjean¹ ·
Geoffrey I. Webb¹

Received: 24 March 2018 / Accepted: 21 January 2019 / Published online: 6 February 2019

© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2019

Abstract

Research into the classification of time series has made enormous progress in the last decade. The UCR time series archive has played a significant role in challenging and guiding the development of new learners for time series classification. The largest dataset in the UCR archive holds 10,000 time series only; which may explain why the primary research focus has been on creating algorithms that have high accuracy on relatively small datasets. This paper introduces Proximity Forest, an algorithm that learns accurate models from datasets with millions of time series, and classifies a time series in milliseconds. The models are ensembles of highly randomized Proximity Trees. Whereas conventional decision trees branch on attribute values (and usually perform poorly on time series), Proximity Trees branch on the proximity of time series to one exemplar time series or another; allowing us to leverage the decades of work into developing relevant measures for time series. Proximity Forest gains both efficiency and accuracy by stochastic selection of both exemplars and similarity measures. Our work is motivated by recent time series applications that provide orders of magnitude more time series than the UCR benchmarks. Our experiments demonstrate that Proximity Forest is highly competitive on the UCR archive: it ranks among the most accurate classifiers while being significantly faster. We demonstrate on a 1M time series Earth observation dataset that Proximity Forest retains this accuracy on datasets that are many orders of magnitude greater than those in the UCR repository, while learning its models at least 100,000 times faster than current state-of-the-art models Elastic Ensemble and COTE.

Keywords Time series classification · Scalable classification · Time-warp similarity measures · Ensemble

Responsible editor: Panagiotis Papapetrou.

✉ Benjamin Lucas
benjamin.lucas@monash.edu

Extended author information available on the last page of the article

1 Introduction

A growing number of time series applications address training from orders of magnitude more series than the largest in the benchmark UCR repository—the 8926 training series ElectricDevices data set. In contrast, the phoneme dataset (Hamooni and Mueen 2014) contains 370,000 series. The satellite dataset (Tan et al. 2017) contains 1,000,000 series. The prior state-of-the-art in time series classification does not scale to such quantities. In 2017, a meticulous study was conducted to compare the behaviour of the state of the art (Bagnall et al. 2017). The authors draw the following conclusions:

1. The state of the art is led by four classifiers that are: Collective of transformation-based ensembles (COTE) (Bagnall et al. 2015), Elastic ensemble (EE) (Lines and Bagnall 2015), Shapelet transform (ST) (Hills et al. 2014) and Bag of SFA symbols (BOSS) (Schäfer 2015).
2. COTE is a special case in that it subsumes two of the other classifiers: it is a large ensemble classifier that includes EE and ST as sub-classifiers; COTE is on average, “clearly superior to other published techniques.”
3. COTE’s runtime complexity is bounded by (a) Shapelet transform, which is $O(n^2 \cdot l^4)$ (Bagnall et al. 2015) for n time series of length l , and (b) the parameter searches for EE, some of which are $O(n^2 \cdot l^3)$. The authors conclude “An algorithm that is faster than COTE but not significantly less accurate would be a genuine advance in the field.”

This is the challenge we tackle in this paper: developing an algorithm that is competitive with the accuracy of the state of the art, but can learn from datasets with millions of time series. We call our algorithm *Proximity Forest*. It is a tree-based ensemble that makes the most of the decades of research into developing consistent similarity measures for time series.

Typical decision trees branch on the value of an attribute. Treating the values at each time stamp as belonging to a single attribute does not work well on time series because the relevant signals are not necessarily aligned by time stamp. Instead, Proximity Forest branches on the *proximity* of a query time series to a set of reference series. ‘Proximity’ is defined by a given (time series) similarity measure and a set of parameters (most time series measures have parameters that are critical to their proper function). Our trees define separating hyperplanes for which the position is supported by time series themselves (whereas a traditional tree would split using a threshold on the value of an attribute). Proximity Forest, as opposed to nearest neighbour approaches, truly abstracts a model from data, which makes it possible to classify with time that is logarithmic with respect to training set size, as opposed to linear time for EE and COTE. Moreover, we will show that we specifically designed its training to scale linearly with the quantity of data, as opposed to at least quadratically for EE, COTE and ST.

Figure 1 shows the accuracy and the training time required by Proximity Forest and EE on our satellite dataset with increasing training set size. Two important elements are illustrated: (1) Proximity Forest scales linearly with training set size while EE scales quadratically; and, (2) Proximity Forest’s classification accuracy on this dataset is substantially better than EE, even when they train on the same quantity of data. For our application, Proximity Forest can learn from 1M time series in 17 h (on 1 CPU)

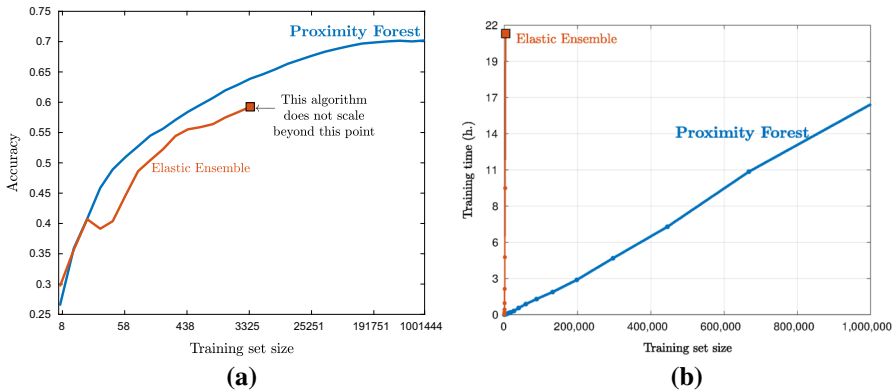


Fig. 1 Comparison of Proximity Forest (in blue) with Elastic Ensemble (in red). **a** Classification accuracy as a function of the size of the training data. **b** Training time as a function of the size of the training data. Note that we take Elastic Ensemble because it is the classifier that prevents scalability of the state-of-the-art COTE ensemble, which includes EE as one of its classifiers (Color figure online)

while it would take over 200 years for EE—a 103,000x speedup. Furthermore, ST and COTE learn slower than EE and thus will have *even larger* training times. Note that EE is a component of COTE and hence sets a lower bound on COTE’s training time. Our discussion will often focus on EE because EE is very similar to our algorithm in that it is trying to leverage existing time series similarity measures.

The virtues of Proximity Forest are not limited to large datasets, however. Our experiments show that it also outperforms EE in classification accuracy on the majority of the datasets of the UCR Archive.

The remainder of this paper is structured as follows: in Sect. 2, we review the state of the art in time series classification, with a particular focus on scalability. We then introduce Proximity Forest in Sect. 3. Our experiments (Sect. 4) show that Proximity Forest (1) outperforms all other scalable algorithms on our case study in both accuracy and training time; and (2) is competitive with the state of the art on UCR data in terms of accuracy. The section ends with a study of Proximity Forest’s parameters and a discussion of how their values vary the results.

2 Time series classification: related work

We present here a non-exhaustive review of the state of the art in time series classification and similar decision tree-based algorithms. We focus on our particular interest in this paper: scalable training and classification.

2.1 Distance-based classification

2.1.1 Distances

Time series have particular properties that have led to the development of specific similarity measures: they are often auto-correlated (the value of the time series at a

timestamp is likely to be close to the ones just before and after), and often include non-linear distortions in the time axis (for example, because the start of the phenomenon of interest is delayed, or because sections of the phenomena are faster or slower). This has rendered typical similarity measures severely flawed and led to the development of specific similarities, of which most have an ability to re-align the series along a common intrinsic time-line. Important measures include Dynamic time warping (DTW) (Sakoe and Chiba 1971, 1978), Derivative DTW (DDTW) (Keogh et al. 2001; Górecki and Łuczak 2013), Weighted DTW (WDTW) (Jeong et al. 2011), Longest common subsequence (LCSS) (Vlachos et al. 2006), Edit distance with real penalty (ERP) (Chen and Ng 2004; Chen et al. 2005), Time warp edit distance (TWE) (Marteau 2009) and Move-Split-Merge (MSM) (Stefan et al. 2013). A more complete description and comparison of these distance measures can be found in Bagnall et al. (2017); Lines and Bagnall (2015); Wang et al. (2013). Note also that most of these distances have parameters of which tuning is critical to their functioning.

2.1.2 Nearest-neighbor approaches

It is most common to classify time series data using Nearest Neighbour classification based on a relative distance (such as those returned from the measures above) (Haghir et al. 2017). In fact, for more than a decade, the NN algorithm combined with the DTW measure was extremely difficult to beat (Wang et al. 2013).

It is important to note here that when researchers mention the use of NN with a time series measure, the measure is not directly applied with a default parameterization, but rather its parameters are first learned on the data, usually by cross-validation.

There are two main issues with NN approaches: (1) the tuning of the measures' parameters is usually quadratic with the size of the training data, and (2) the classification is at least linear with the size of the training data. Both of these issues are further compounded by the fact that most measures have a computational complexity that sits between linear and quadratic with the length of the series.

To alleviate the second issue of scalable classification, targeted techniques have been developed. Data reduction techniques aim at simplifying the training database without penalizing the classification quality; either by directly removing objects from the original database (Pekalska et al. 2006; Ueno et al. 2006) or by summarizing the database and replacing sets of time series with representatives using average time series (Petitjean and Gançarski 2012; Petitjean et al. 2014, 2016; Marteau 2016). Indexing is more difficult on time series than it is on traditional data, mostly because time series measures do not obey the triangular equality (at most obeying a relaxed p -triangular inequality Lemire 2009), which makes exact pruning very inefficient (however, a general approach to improving the efficiency of NN searches in non-Euclidean space is available in Lifshits 2010). To perform exact indexing, the main research effort has been put onto developing lower bounds (and mostly for DTW) (Keogh et al. 2006; Lemire 2009). Recently, impelled by the motivating application of Earth observation data analytics, we have developed an algorithm for approximate and efficient NN search under DTW (Tan et al. 2017), an algorithm using the idea of a hierarchical k -means clustering.

As mentioned previously, Elastic ensemble (EE) (Lines and Bagnall 2015) is a recent state-of-the-art time series classifier. It is an ensemble of 11 NN classifiers, each learned with a different time series measure (with their parameters tuned accordingly). The EE algorithm has played a significant role in the design of Proximity Forest and will be discussed at greater depth in Sects. 3.1 and 3.2.

2.2 Approaches that learn features

The following approaches construct an abstraction of the training dataset by learning features that represent the classes in the time series.

2.2.1 Shapelets

The aim of shapelet algorithms is to find subseries (or consecutive subsets of time series) that can help discriminate between the different classes. To classify a time series, the learned shapelet is placed at the best position in the time series (usually under Euclidean distance), and the ‘matching’ of the shapelet to the time series correspond to its distance at this best position. The original shapelet-classifier (Ye and Keogh 2011) inserted this algorithm at the node of a decision tree as a splitting criterion. This algorithm has a high training complexity ($O(n^4)$) due to the large number of candidate shapelets and the repeated scanning of the data. Subsequent research has focused on optimising the original algorithm to address both classification accuracy and scalability, notably Fast Shapelets (Rakthanmanon and Keogh 2013), Learning Time Series Shapelets (Grabocka et al. 2016) and Shapelet Transforms (Hills et al. 2014).

Shapelet transforms (ST) is a current state-of-the-art classifier that identifies the best k shapelets in a single scan of the data (the number of shapelets can be reduced afterwards). The data is then transformed by defining an attribute to represent each shapelet with the value being the (usually Euclidean) distance between the shapelet and the best position in the time series. The transformed dataset can now be used with any classifier or ensemble of classifiers (such as in Bagnall et al. 2015). While ST is considered a state-of-the-art classifier, it has little potential to scale to large datasets given its training complexity of $O(n^2 \cdot l^4)$.

2.2.2 Bag of words approaches

Bag of words are similar to shapelets in that they start by identifying exemplar subseries in the data to discriminate between classes. However rather than finding the similarity to the relative best positions in a time series, bag of words approaches differentiate classes by the relative frequency of the subseries. To calculate these frequencies, the algorithms discretise the values into a series of symbols, assigning letters to each subseries, and thus representing the original time series as ‘words’. Notable approaches are the Bag of patterns (Lin et al. 2012); the Symbolic aggregate approximation vector space model (SAX-VSM) (Senin and Malinchik 2013); and the Bag of SFA symbols (BOSS) (Schäfer 2015), which is currently considered state of the art.

The BOSS algorithm transforms the time series into a word using a symbolic Fourier approximation (SFA) (Schäfer and Höggqvist 2012) thus making it robust to noise and delivering a high classification accuracy. It is however of limited use on large datasets as it has a high training complexity $O(n^2)$ (Bagnall et al. 2017). The authors identified this as a weakness and subsequently produced similar approaches with improved scalability, the Bag of SFA symbols in Vector space (BOSS-VS) (Schäfer 2015). The same authors recently proposed WEASEL (Schäfer and Leser 2017), which improves on the computation time of BOSS and on the accuracy of BOSS-VS, but has a very high memory complexity (our experiments will show that it doesn't scale beyond 10,000 time series). In this way, WEASEL is more optimised for speed on small datasets than for scalability.

2.3 Ensemble approaches

Ensemble approaches are combinations of multiple classifiers. Each contributing algorithm can be weighted to maximize classification accuracy, while the time complexity is that of the slowest constituent. Some of these approaches have been discussed above as they are based around one main type of classifier, for example EE and ST.

The Collective of transformation-based ensembles (COTE) (Bagnall et al. 2015) is an ensemble comprising 35 classifiers across four time series domains: time, frequency, change and shapelet transformation. For the time domain, COTE uses the 11 distance measures of EE, while in the other three domains, classifiers are recruited from outside time series classification— k -nearest neighbours, naive Bayes, decision trees, random forest, rotation forest, support vector machines (two models) and a Bayesian network approach. On the benchmark UCR datasets, COTE has the highest average classification accuracy of all current approaches. However, its time complexity is bound by that of the Shapelet transform, which is $O(n^2 \cdot l^4)$ and the parameter searches for the elastic distance measures (EE), some of which are $O(n^2 \cdot l^3)$.

2.4 Decision tree approaches

A number of decision tree approaches have been developed for time series classification.

Time series forest (TSF) (Deng et al. 2013) first derives summary features for all time series by dividing them into intervals and summarising each interval by its mean, standard deviation and gradient. Then a Random Forest-like strategy is employed to select between a random subset of these features at each node in each of an ensemble of trees. A novel selection criterion is used that considers both entropy gain and the margin by which a feature separates the classes. This continues until the entropy gain ceases to improve, at which stage the node is defined as a leaf. TSF has been shown to be a reasonably accurate classifier: its accuracy ranks behind EE and DTW without being significantly worse (Bagnall et al. 2017). However, its main virtue is computational efficiency. TSF learns in $O(n \log(n) \cdot l \cdot k)$ for a forest of k trees built from n series of length l , which is a much lower complexity than the current state of the art.

Generalized random shapelet forest (gRSF) (Karlsson et al. 2016) extracts a shapelet from a randomly chosen time series and finds the distance between this time series and each other time series. The data is then split according to whether it is above or below a threshold distance to the representative shapelet. This is applied recursively until the node is either pure or the number of instances remaining at a node is less than 3. As mentioned in Sect. 2.2.1, the main pitfall of shapelet-based methods is the high computational cost of finding candidate shapelets and comparing shapelets to other time series. The gRSF minimises this issue by randomising many of the model choices—a candidate shapelet is generated from a randomly chosen time series by choosing a random starting point and random length, this is repeated r times and the best candidate is chosen for a given split. The resulting algorithm has accuracy competitive with Learning Time Series Shapelets and better than DTW.

A number of approaches have been developed that form decision trees where splits are based on **similarity to chosen exemplars** (Balakrishnan and Madigan 2006; Douzal-Chouakria and Amblard 2012; Yamada et al. 2003). One strategy is to **select a single exemplar and then choose a cut point on a similarity measure with respect to that exemplar. Series with similarity scores lower than the cut point follow one branch and the remaining examples follow the other.** The other strategy is to select multiple exemplars, one associated with each branch. Series follow the branch with whose exemplar they are most similar. These approaches are hampered by the high computational complexity of their search for exemplars at each node. **Similarity Forests** (Sathe and Aggarwal 2017) and **Comparison-based Random Forests** (Haghiri et al. 2018) generalise this idea to attribute-value data with random selection of exemplars and developed forests of such trees. Similarity Forests add a cutoff value on the difference in the distance between the two exemplars, and optimizes that cutoff value based on weighted Gini. The idea of using similarity as the splitting criterion in tree structures has also been successfully used for *indexing* of regular tabular data (www.cs.ubc.ca/research/flann/, Bernhardsson 2013; Sathe and Aggarwal 2017) and of time series with DTW (Tan et al. 2017).

3 Proximity forest

In this section, we present our novel algorithm for time series classification: Proximity Forest. We start by highlighting why there is a need for a new time series classifier. We then present our model and the two key algorithms (1) how to learn a Proximity Forest and (2) classifying with a Proximity Forest. We conclude this section with some comments about its complexity.

3.1 Why do we need a novel time series classifier?

The previous section highlighted that the last decade has seen numerous classifiers and distance measures specifically designed for time series classification. Based on this, one could wonder why there is a need for a novel algorithm; the answer is simple: most state-of-the-art algorithms do not scale to large time series datasets. We have

seen that some do not scale in the learning phase (ST, EE, COTE). Others require a scan of the training database to perform each classification (EE, COTE). Those that do scale to medium-size datasets, such as BOSS-VS, compromise accuracy in order to do so (as we will show for both our case study and for UCR datasets). Throughout the development and advancement of much of the current state of the art, scalability has usually been secondary to accuracy. This is because time is not a significant issue when considering data with only few time series. However, a growing number of modern applications consist of hundreds of thousands to millions of time series. These applications require a classifier that is both accurate *and* scalable in both learning and classification.

BOSS-VS is a classifier that appears to have developed with a focus on scalability. However, as we will see in Sect. 4, its accuracy ranks some 30% points lower in our case study, and therefore is not competitive with the accuracy of the state of the art.

While COTE is currently *the* state of the art in terms of accuracy, its learning phase is bound by the runtime complexity of both ST and EE. On our 1M dataset—and as depicted in Fig. 1—the sole learning phase of COTE associated to training EE would require 73,000 days, or 200 years. This is even more startling knowing that the series in this dataset are very short with only 46 timestamps.

The large runtime complexity of COTE is largely due to the fact that EE does not abstract much information during the learning phase, and therefore has a significantly greater number of processes to complete during testing. A corollary of this is that a distance-based classifier that learns faster than EE for the same level of accuracy would also present an improvement to COTE. It is for this reason that our design of algorithm incorporates many elements of EE—11 distance measures and similar parametrisation—and why our experiments provide a direct comparison of Proximity Forest against EE.

We therefore argue that the need for a scalable and accurate classifier has not yet been met. We incorporate three critical elements into the design of our novel approach:

1. We make the most of over 30 years of research into designing consistent measures for time series.
2. We specifically design our ensemble to have a high variability between the different individual classifiers. This results in an improved overall classification accuracy over a single classifier, based on the principle of ensemble methods. In general, averaging the predictions of multiple models each having high variance and low bias results in an ensemble classifier with a lower total error than any single classifier. **This is analogous to how a Random Forest model, another ensemble of decision trees, will only learn from a fraction of the available features for each individual node in order to introduce variability between the trees** (Breiman 2001; Ho 1995). This observation is important, because we did not design the learning of an individual tree to maximize its accuracy; if we had wanted to design a single tree model, we would have made different design choices. We designed the learning of individual trees so that the overall classification performance is maximised.
3. We design Proximity Forest to be extremely scalable with an average-case learning complexity of $O(n \log(n) \cdot l^2)$ and a classification complexity of $O(\log(n) \cdot l^2)$ per tree for n training time series of length l . This contrasts with the state-of-the-art

learning in $O(n^2 \cdot l^3)$ (Elastic ensemble) or $O(n^2 \cdot l^4)$ (Shapelet transform, COTE). Again here, we might have made different choices if scalability wasn't a design objective.

To achieve scalability we employ tree-based classifiers. These are scalable due to their use of a divide-and-conquer strategy. At each level the data are divided into multiple subsets, as result of which the trees are on average of depth $O(\log n)$, hence increasing sublinearly in depth relative to training set size.

Our use of decision trees for time series classification is not novel in itself. Trees are attractive due to their divide and conquer methodology and resulting potential for efficient learning and classification. Previous implementations, however, have lacked competitiveness in accuracy (Deng et al. 2013; Douzal-Chouakria and Amblard 2012; Yamada et al. 2003) or time (Balakrishnan and Madigan 2006; Douzal-Chouakria and Amblard 2012; Yamada et al. 2003).

To achieve scalability we merge the strategy of learning decision trees where splits are based on similarity to chosen time series exemplars (Balakrishnan and Madigan 2006; Douzal-Chouakria and Amblard 2012; Yamada et al. 2003) with the strategy of forming forests of such trees in which the exemplars are chosen at random (Sathe and Aggarwal 2017). To this amalgam we add the critical ingredient of stochastic selection between a large range of similarity measures, which both reduces bias and provides a beneficial increase in variance between ensemble members.

3.2 How to learn a Proximity Forest?



We seek to learn a Proximity Forest from a training set comprising n labeled time series, each of which is of length l , where the labels are integers from 1 to c .

A Proximity Forest is an ensemble of k Proximity Trees. A Proximity Tree is similar to a regular decision tree, but differs in the tests applied at internal nodes. Whereas a regular decision tree applies a test based on the value of an attribute (e.g. if height > 160 cm, follow the left branch, otherwise follow the right branch), **each branch of an internal node of a Proximity Tree has an associated exemplar and an object follows the branch corresponding to the exemplar to which it is *closest* according to a parameterized similarity measure**. We will see later how the exemplars and measures are chosen. A tree is either a leaf or an internal node.

An internal node has two fields, *measure*, a function $object \times object \rightarrow \mathbb{R}$, and *branches*, a vector of branches. Each branch has two fields, a time series (exemplar) and a tree to which an object is passed if it is nearer to the branch's exemplar than any other (subtree).

If all data reaching a node has the same class, i.e. is pure, the *create_leaf* function creates a new leaf node and assigns this class label to its field *class*. This label is then assigned to any query time series reaching this leaf during the testing phase.

How do we choose the splitting criteria? A Proximity Tree creates, at each node, one branch for each class that exists in the data it receives from its parent. These exemplars are chosen uniformly at random among each class. The parameterized similarity measures are also chosen uniformly at random among a pool that will be

described below after we have given the main overview of the algorithm. We will detail, after the main algorithm, how it is possible to *learn* with randomized trees.

Algorithm 1 presents the algorithm for learning a single tree. Each node is constructed recursively from the root node down to the leaves. If the data at the node is pure—i.e., all data belongs to the same class—then the node becomes a leaf and the recursion finishes.

At each node, a pool of r candidate splits are evaluated (Algorithm 2). For each candidate, a parameterised measure is chosen uniformly at random among a pool of such measures. We then select an exemplar for each class represented at the node and pass the data down the branches by finding the closest exemplar (one per class) for each time series in the data using the split's distance measure.

Once each candidate split has been created, we then select the candidate that maximizes the difference between the Gini impurity of the parent node and the weighted sum of Gini impurity of the child nodes. We then call the construction of the tree recursively on each branch for the successful candidate; this constructs all subtrees. When this is done, the tree is constructed.

Increasing the number of candidate splits per node will lead to an improvement of the quality of each split. However, it will also lead to an increase of the training time. The choice for the value of r will be discussed later in Sect. 4.3.2.

The case of $R = 1$: is selecting at random still 'learning'? One might wonder what the tree is actually learning when one only considers a single candidate ($R = 1$). In that case, no selection of 'the best possible split' is performed. It is interesting to note that choosing splitting criteria independently of the output value has been studied before, a key example being Extremely randomized trees (Geurts et al. 2006). In that work, they showed that splitting completely at random still ensures consistency (tending to Bayes optimal error as the data tends to infinity). The main reason is that the exemplars are not random points in the input space. They are sampled from the data distribution of each class. In consequence, the trees *are* still learning an abstraction of the data, using the trees as a density estimator (Ting et al. 2016).

We depict in Fig. 2 a graphical representation of a simple split obtained on the Trace dataset. It is interesting to see that in Euclidean space, the splitting criterion is actually forming a hyperplane that is equidistant to the exemplars. Note that this intuition is more complex for time series measures, because most of them do not have properties of a metric (Lemire 2009). The scatter plot depicts each time series as a dot in this space, with the x-axis representing distance to the first exemplar and the y-axis distance to the second.

How to choose the parametrised measure on which to split? The parametrised distance measure gives a measure of the similarity between the exemplar time series. For each candidate split at each node, the algorithm chooses a distance measure at random from the following 11 distance measures used by the Elastic ensemble (EE) learner that we described above: Euclidean distance (ED); Dynamic time warping using the full window (DTW); Dynamic time warping with a restricted warping window (DTW-R); Weighted dynamic time warping (WDTW); Derivative dynamic time warping using the full window (DDTW); Derivative dynamic time warping with a restricted warping window (DDTW-R); Weighted derivative dynamic time warping (WDDTW); Longest

Algorithm 1: build_tree(D, Δ, R)

Input: D : a time series dataset
Input: Δ : a set of parameterized distance measures
Input: R : number of candidate splits to consider at each node
Output: T : a Proximity Tree

```

if is_pure( $D$ ) then
  | return create_leaf( $D$ )
  // create tree, to be returned, represented as its root node

 $T \leftarrow$  create_node()

// Creating  $R$  candidate splitters
 $\mathcal{R} \leftarrow \emptyset$ 
for  $i = 1$  to  $R$  do
  |  $r \leftarrow$  gen_candidate_splitter( $D, \Delta$ )           // generate random splitter
  | Add splitter  $r$  to  $\mathcal{R}$ 

// select best splitter using measure  $\delta^*$  and exemplars  $E^*$ 
 $(\delta^*, E^*) \leftarrow \arg \max_{r \in \mathcal{R}} \text{Gini}(r)$ 

 $T_\delta \leftarrow \delta^*$                                      // retain measure for root node of  $T$ 
 $T_B \leftarrow \emptyset$                                //  $T_B$  will store the branches under root node of  $T$ 
foreach exemplar  $e \in E^*$  do
  | //  $D_e^*$  is the subset of  $D$  that are the closest to  $e$  based on  $\delta^*$ 
  |  $D_e^* \leftarrow \left\{ d \in D \mid \arg \min_{e' \in E^*} \delta^*(d, e') = e \right\}$ 
  |  $t \leftarrow$  build_tree( $D_e^*, \Delta, R$ )               // build subtree for that branch
  | Add branch  $(e, t)$  to  $T_B$                        // a branch is a pair (exemplar, sub-tree)

return  $T$ 

```

Algorithm 2: gen_candidate_splitter(D, Δ)

Input: D : a time series dataset
Input: Δ : a set of parameterized distance measures to sample from
Output: (δ, E) : a parameterized distance measure and a set of exemplars

```

// sample a parameterized measure  $\delta$  uniformly at random from  $\Delta$ 
 $\delta \xleftarrow{\sim} \Delta$ 

// select one exemplar per class to constitute the set  $E$ 
 $E \leftarrow \emptyset$ 
foreach class  $c$  present in  $D$  do
  |  $D_c \leftarrow \{d \in D \mid \text{class}(d) = c\}$            //  $D_c$  is the data for class  $c$ 
  |  $e \xleftarrow{\sim} D_c$            // sample an exemplar  $e$  uniformly at random from  $D_c$ 
  | Add  $e$  to  $E$ 

return  $(\delta, E)$ 

```

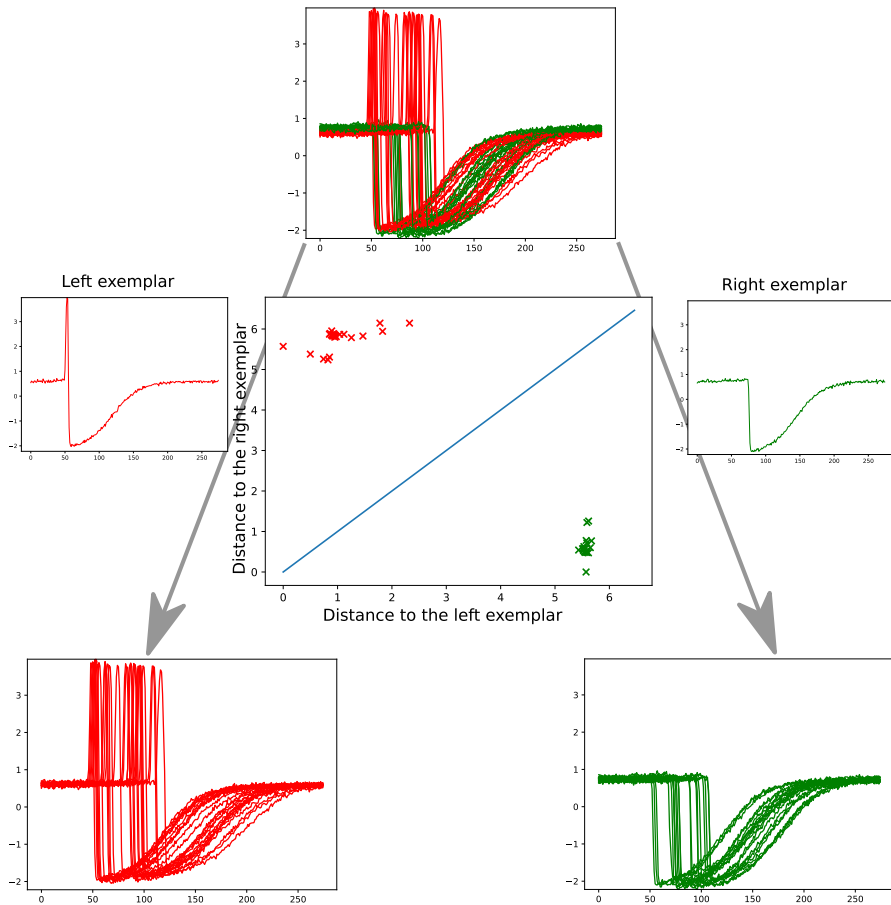


Fig. 2 Visual depiction of the root node for the ‘Trace’ dataset (simplified to 2 classes). The top chart represents the data at the root node (one colour per class) while the data at the bottom left and right represent the data once split by the tree. The two time series in the middle left and right are the exemplars on which the tree is splitting. The scatter plot at the center represents the distance of each time series at the root node with respect to the left and right exemplars (resp. x- and y-axes) (Color figure online)

common subsequence (LCSS); Edit distance with real penalty (ERP); Time warp edit distance (TWE); and, Move-Split-Merge (MSM). Randomising the choice of distance measure is a deliberate decision to introduce variability between each tree, for the reasons stated earlier.

Once a distance measure is chosen at random, it is then parametrised. The parametrisations are addressed in turn. They are deliberately chosen to mimic as closely as possible the EE algorithm. Even though better values might be chosen here, we mimic EE’s parameterization to allow direct comparison. Euclidean distance, full DTW, and full DDTW distances have no parameters to select. DTW-R and DDTW-R only require a warping window parameter that is chosen uniformly at random in $[0, \lfloor \frac{l+1}{4} \rfloor]$ (thus allowing a warping of elements at most $\frac{l}{2}$ apart). WDTW and WDDTW requires

also one parameter to select that it is used into the weighted value g to control the level of penalization between two different time stamps—we use $g \sim U(0, 1)$. The parametrisation of ERP is a distance threshold that controls for how close elements have to be to be considered similar; we sample it uniformly at random in $[\frac{\sigma}{5}, \sigma]$, with σ being the standard deviation of the data. LCSS has as first parameter the same distance threshold value (which is sampled in the same way), and has a second parameter—the warping window size—which is chosen in the same way as for DTW-R. TWE has two parameters γ and λ which respectively control for the stiffness and penalty value in the alignment. Following (Marteau 2009), λ is sampled at random from $\cup_{i=0}^9 \frac{i}{9}$ and γ following at random from the exponentially growing sequence $\{10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}, \dots, 1\}$, resulting in 100 possible parameterizations. The final measure, MSM, has a single parameter which is sampled from an exponential sequence similar to the one for γ in TWE with 100 values ranging from 10^{-2} to 10^2 , as recommended in Stefan et al. (2013).

Choosing the parameter at random has a twofold effect: (1) it skips the cross-validation step which has a quadratic complexity; and (2) it introduces variability between trees, which provides superior learning through lower-biased trees and ensembling. In the following experiments we will show that Proximity Forest is not only orders of magnitude faster than EE, but that its accuracy also ranks higher than EE.

3.3 Classifying with a Proximity Forest

The process of classification for a single Proximity Tree is detailed in Algorithm 3: a query time series begins at the root node and the distance from the query to each of the exemplar time series is calculated, by using the node's distance measure and exemplars selected when constructing the tree. The query time series is then passed down the branch of the exemplar to which it is nearest. The query time series then traverses down the tree by repeating this process until it reaches a leaf, where it is assigned the class represented by that leaf. This process is repeated for each tree constructed as part of the forest. A Proximity Forest then uses majority voting between its constituent Proximity Trees.

Algorithm 3: classification(Q, T)

Input: Q : Query Time Series

Input: T : Proximity Tree

if is_leaf(T) **then**

return majority class of T

 // find the branch with exemplar closest to Q using measure T_δ

$(e, T^*) \leftarrow \arg \min_{(e', T') \in T_B} T_\delta(Q, e')$

return classification(Q, T^*)

 // recursive call on subtree T^*

3.4 Comparative complexity analysis

During the training phase, at each node, let us assume that n' data points are present at the node. We first scan it once taking $O(n')$ time to split the data into c groups, one for each class c present at the node.

We then generate r candidate splits, i.e., r sets of exemplar time series. For each such candidate set, we sample $c' \leq c$ exemplar time series, i.e., one time series for class represented among the n' time series available at the node—this is done in $O(1)$ given that the data is already organised by class. For the candidate split to be operational, we also require a parameterized measure to use to compare against these c' exemplars. Most of the parametrized measures can be chosen in $O(1)$, except for LCSS and ERP which calculate the standard deviation in $O(n' \cdot l)$ for data at the node while selecting the parameter.¹

We now have r candidate splits that are ready to be evaluated. We now wash the n' time series down the branches for all candidate splits. This is done by comparing each time series to the c' exemplars, each comparison taking from $O(c \cdot l)$ to $O(c \cdot l^2)$. Overall, this takes $O(n' \cdot c' \cdot l^2)$. If $r = 1$, the training process at this node is finished and we call the training function recursively for each of the c' children nodes. If $r > 1$, we calculate the Gini coefficient for each of the r candidate splits in $O(c^2)$, keep the best one, and delete other candidate splits.

As the total number of examples that reach any of the nodes at a single given level cannot be greater than the total number of examples, n , the total computation per level of the tree is thus $O(n \cdot r \cdot c \cdot l^2)$. In the worst case, the majority of the training data at each level will pass down a single branch and the depth of the tree will be $O(n)$, resulting in a worst training time complexity of $O(n^2 \cdot r \cdot c \cdot l^2)$. However, as the exemplars are following the class distribution, unless the data are in some way degenerate (for example if one class comprises only outliers), the average tree depth can be expected to be $O(\log n)$. In practice it will often be much smaller, because, unlike typical divide-and-conquer approaches, the tree terminates as soon as a node is pure rather than having to separate each individual object. Thus, for non-degenerate data we can expect average case training time complexity of $O(n \log(n) \cdot r \cdot c \cdot l^2)$ for a single tree and thus $O(k \cdot n \log(n) \cdot r \cdot c \cdot l^2)$ for a full Proximity Forest comprising k Proximity Trees.

The experiments presented in the next section will include runtimes and comparison to current state-of-the-art algorithms. These confirm that this expected average case quasi-linear complexity with respect to data quantity is borne out in practice.

During classification, a time series of length l will pass through an average of $\log n$ nodes on each of the k trees. At each node, the distance to at most c exemplars must be computed. For each of these distance computations, the complexity will again depend upon the chosen distance measure; the fastest being $O(l)$ and the slowest $O(l^2)$. Thus, the resulting average case complexity is $O(k \cdot \log n \cdot c \cdot l^2)$.

¹ Note that these parametrisations can be performed in constant time also if the data are z -normalized, which is the case for all UCR datasets.

4 Experiments

This section describes the experiments that evaluate our Proximity Forest. We start with the satellite image time series (SITS) dataset, a (very) large time series dataset describing the evolution of the Earth as pictured every five days by a high-resolution satellite. This dataset is an example of the large time series datasets that motivate the need for a new time series classification algorithm, as no current state-of-the-art approach scales to this magnitude. Conversely, there are classifiers designed for scalability, namely BOSS-VS, that compromise classification accuracy to do so. The first experiments presented in this section use the SITS dataset to demonstrate the ability of Proximity Forest to be both scalable *and* accurate. The second section assesses the Proximity Forest on the datasets of the UCR time series classification repository (Chen et al. 2015), the benchmark in the field. It demonstrates that the classification accuracy of Proximity Forest is competitive with the current state of the art. The final section discusses other considerations surrounding Proximity Forest, such as the effect of varying the number of trees, and the standard deviation of the results.

It should be mentioned that throughout the following experiments we have emphasized a comparison with EE. This is because it is viewed as the closest relative to Proximity Forest amongst the current state of the art, given that neither method includes data transforms or shapelets. It is also the constituent of COTE that bounds its learning time and therefore any improvement over the runtime of EE, for the same classification accuracy, would also equate to an improvement on COTE, the current leader in the field.

To facilitate others to build on our work, as well as to ensure reproducibility, we have made our [code and the full raw results available](https://github.com/fpetitjean/ProximityForest/) at <https://github.com/fpetitjean/ProximityForest/>.

4.1 Case study: satellite image time series dataset

The SITS dataset contains approximately 1 million time series with a train-test split of approximately 90–10%.² Each time series has a length of 46 and is labeled as one of 24 possible land-use classes (e.g. ‘wheat’, ‘corn’, ‘plantation’, ‘urban’). Here the labeled data has been extracted from three sources: (1) ground field campaigns for most of the vegetation classes, (2) farmer’s declaration to complete the data for some crop classes, and (3) existing map for the urban areas.

The experiments presented in this section were performed on this dataset, comparing the performance of Proximity Forest against three competitors: BOSS-VS (designed for scalability), WEASEL (designed for speed and quality), and EE (designed for quality). We use 5 runs for each experiment of Proximity Forest and 1 run of each of the competitors—as their results are deterministic. Throughout this experiment, we use 100 trees; we will see in Sect. 4.3 that this gives a good tradeoff between accuracy and computational time/memory. Although we are mainly assessing the scalability, we will also have a quick look to the accuracy.

² The split ensures that no 2 times series come from the same plot of land.

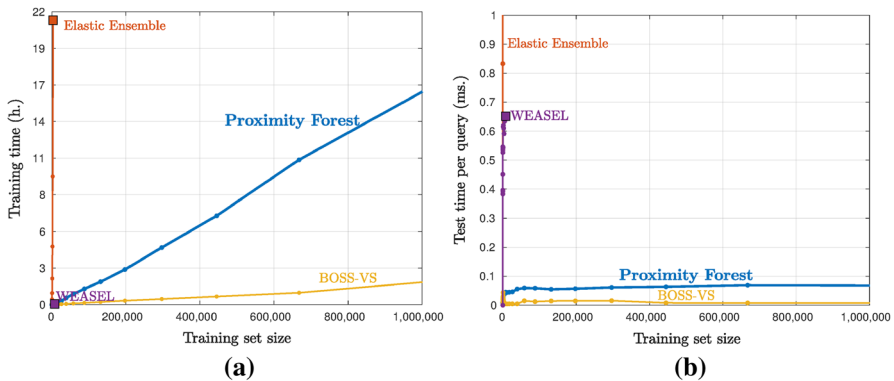


Fig. 3 Training time (a) and testing time per query (b) as a function of training set size for Proximity Forest, EE, WEASEL and BOSS-VS

4.1.1 Training scalability

To assess scalability, we train and test each algorithm on subsample data with increasing training set size, allowing training time, testing time and accuracy to be measured as a function of training size. Figure 3a shows training time against training size for each of the 4 algorithms.

Versus EE. First, it is evident that Proximity Forest presents a notable saving in training time over EE, confirming that it trains in linear time rather than the quadratic time for EE. Even for a small training set of about 2000 time series, learning an EE model took about 10 h, compared to Proximity Forest's 79 sec. Fitting a quadratic curve through both EE and Proximity Forest is quite informative: EE returns a quadratic component of 6.3 while Proximity Forest only -8.10×10^{-6} , clearly highlighting both the quadratic complexity of EE, and also that Proximity Forest is in practice very close to its theoretical average complexity and scales quasi-linearly with n .

Versus WEASEL. WEASEL is very fast but its memory footprint did not allow it to scale beyond 8000 time series even when given 64 GB of RAM. This clearly highlights the difference with BOSS-VS: we can see that WEASEL was not developed for scalability, but rather for speed on small datasets.

Versus BOSS-VS Proximity Forest trains slower than BOSS-VS for a given training size, however this is counteracted by the low accuracy of BOSS-VS discussed below.

4.1.2 Testing scalability

Figure 3b shows testing time against training size for each of the 4 algorithms. The story here is very similar to that of training: it confirms the way Proximity Forest scales logarithmically with training set size, while EE must scan the full database many times. Here again, WEASEL becomes infeasible to apply with relatively small quantities of training data. Proximity Forest and BOSS-VS require respectively 0.0679 ms and 0.0077 ms to classify a time series with a model trained on 1M time series.

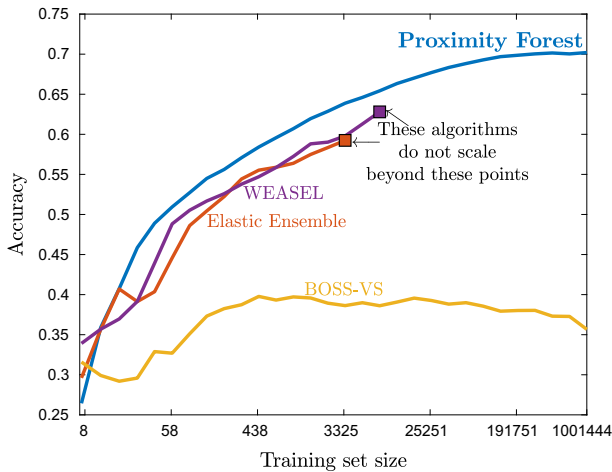


Fig. 4 Accuracy as a function of training set size for Proximity Forest, EE, WEASEL and BOSS-VS

4.1.3 Is proximity forest accurate and scalable?

We have now seen that Proximity Forest is highly scalable and only beaten by BOSS-VS in terms of training time. We will now study how its accuracy scales with training set size. The main results are presented in Fig. 4 which plots the accuracy as a function of training set size for Proximity Forest, EE, WEASEL and BOSS-VS.

The first element to observe is that Proximity Forest obtains greater accuracy than the competitors for large training sets. WEASEL and EE become infeasible to apply at relatively small data quantities and BOSS-VS—which is faster than Proximity Forest—does not learn effective classifiers on this dataset. With 63.8% accuracy at 3400 training set size, this is 26.3 percentage points more accurate than BOSS-VS, and 4.6 and 4.7 percentage points more accurate than WEASEL and EE, respectively. Such differences are substantial in a problem comprising 24 classes.

Moreover, Proximity Forest is more accurate than the other algorithms from 500 training instances upwards. This is not surprising, as trees usually have a better control over variance than NN algorithms, because of their higher bias and abstraction capabilities. Proximity Forest thus appears to be both accurate and highly scalable. We will show in the next subsection that this result holds also on the benchmark UCR archive.

4.2 Experiments on the UCR archive

In this section, we study the behavior of Proximity Forest on the 85 datasets of the traditional UCR archive (Chen et al. 2015). It is useful to remember here that our aim is not to show that Proximity Forest is more accurate than the state of the art, but only that it is competitive while being highly scalable. We compare the mean error-rate of Proximity Forest to the error-rates on the standard train/test split for the state of the art, as tested in Bagnall et al. (2017). We average Proximity Forest results over 10

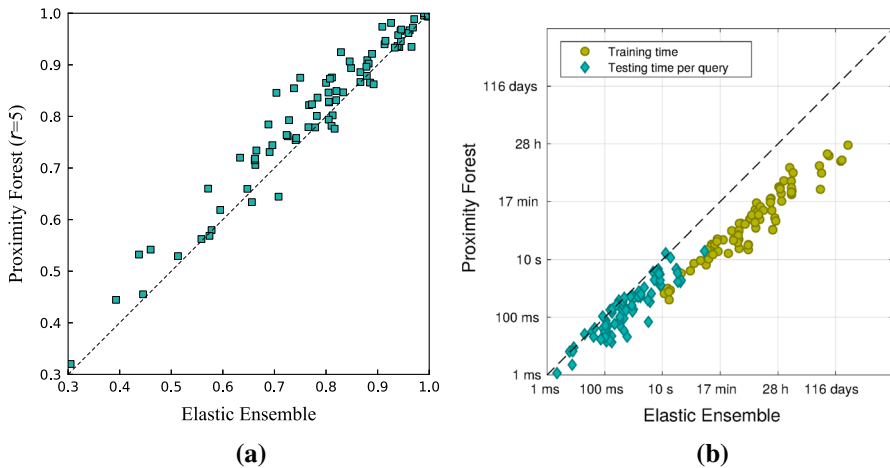


Fig. 5 Comparison of Proximity Forest and Elastic Ensemble classifiers on UCR datasets in terms of **a** accuracy and **b** training and testing times in log scale

runs for each experiment. We compare Proximity Forest to five classifiers currently representing the state-of-the-art—DTW-R, COTE, EE, ST and BOSS³. The Proximity Forest results are obtained for 100 trees with selection between 5 candidates per node. A detailed discussion about the Proximity Forest parameters will be performed in Sect. 4.3.

We first show the comparison with Proximity Forest's closest relative, EE. Figure 5 provides scatter plots of the relative accuracy, total training time and total testing time of each of these classifiers. Each point represents a different UCR dataset. Figure 5a shows that Proximity Forest is more accurate on 60 datasets and less accurate on only 11 datasets, with 14 ties. Moreover, for many datasets Proximity Forest is substantially more accurate than EE.

Figure 5b demonstrates that Proximity Forest has several orders of magnitude advantage in training time. When considering testing time, Proximity Forest has greater test time per query than EE for 12 datasets, the majority of which are small datasets (i.e. less than 50 training instances). The largest such difference is observed for the Phonemes dataset for which Proximity Forest takes about 17 sec per query compared to 13 sec per query for EE. In contrast, the test time for Proximity Forest is much smaller than EE for the biggest datasets (i.e. more than 800 training instances). For example, the biggest test time difference is for the HandOutlines dataset for which Proximity Forest takes about 19 sec per query compared to 286 sec per query for EE.

The commonly accepted method to compare multiple classifiers over multiple datasets is by average ranks. For each dataset, we rank the classifiers and then calculate the average of each classifier's ranks across all datasets. When comparing 6 algorithms over 85 datasets, Demšar (2006) shows that for the rankings to be significantly differ-

³ It should be highlighted that the results presented here are for the original BOSS algorithm, and not the BOSS-VS discussed above in the SITS experiments. BOSS-VS is a scalable variation of BOSS, where concessions are made to accuracy in favor of training time. The original BOSS is therefore more competitive in this section.

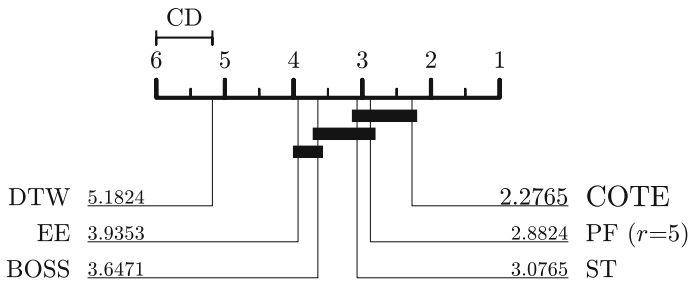


Fig. 6 Critical difference diagram for five state-of-the-art classifiers and Proximity Forest (PF) with 5 candidates

ent at level $\alpha = 0.05$, the critical difference (CD) between the average ranks has to be greater than:

$$CD = q_{0.05}(A) \cdot \sqrt{\frac{A(A+1)}{6 \cdot N_d}} = 2.850 \cdot \sqrt{\frac{42}{510}} \approx 0.82 \quad (1)$$

The average ranks and critical difference are presented in Fig. 6; the critical difference of 0.82 is displayed by the black line. It can be seen that COTE ranks highest (average rank of 2.28), which is to be expected considering it incorporates the other state-of-the-art algorithms. However, COTE is not ranked significantly higher than Proximity Forest (average rank of 2.88) or ST (average rank of 3.08). Proximity Forest is ranked second. Its rank is not significantly different to COTE, ST or BOSS, but it is ranked significantly higher than both EE and DTW. This affirms Proximity Forest as a classifier with accuracy competitive with the state of the art.

Proximity Forest is the most accurate classifier for 22 of the 85 datasets. However, there is no obvious commonality between these datasets to suggest conditions under which the algorithm is likely to excel. The detailed accuracy results for Proximity Forest and the five state-of-the-art algorithms are shown in “Appendix A”.

4.3 Parameters of Proximity Forest

Proximity Forest has two main parameters that merit further investigation. We first explore the sensitivity of accuracy to the number of trees in each ensemble. Then, we discuss the influence of the number of candidates assessed at each node. A third design choice, random selection of similarity measure per tree as opposed to per node, is explored in “Appendix B”.

4.3.1 On the choice of the number of trees

The number of trees is the first parameter of the Proximity Forest algorithm with the optimal value being large enough to provide competitive accuracy, yet small enough

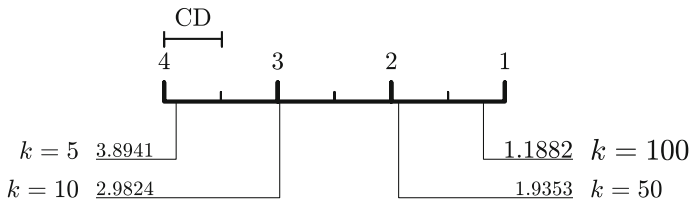


Fig. 7 Critical difference diagram for Proximity Forest with 5, 10, 50 or 100 trees

not to create excessive computational expense. The UCR datasets experiments outlined above were repeated with values of 5, 10, 50, and 100 trees to analyse how many trees were required to meet our needs. Here, the number of candidates r has been fixed to 1. The Proximity Forest results are averaged over 50 runs. Figure 7 presents the critical difference diagram for accuracy and different number of trees. As expected, the more trees the higher the average accuracy: models with 100 trees had an average rank of 1.19 compared to 1.93, 2.98 and 3.89 for models with 50, 10, and 5 trees respectively. The difference between the highest ranked models are large enough to say that models with 100 trees are significantly better than models with 50 trees at the level of alpha equals 0.05.

Figure 8 compares the classification accuracy for 100 trees against 10 and 50 trees by representing them as a ratio of their error rates. Each point represents a single dataset. This shows that having 100 trees is better on most datasets. Moreover, the fact that the data is gathered close to the line with equation $x = 1$ shows that it is unlikely that more trees would provide a very significant improvement, because the ratio of error-rates between 100 and 50 is already close to 1 (i.e., the errors are only slightly reduced). We have not experimented with forests comprising more than 100 trees as we felt the computational demands outweighed the expected benefits for our large set of experiments. Memory, training time and testing time all scale linearly with the number of trees, which means that doubling the number of trees doubles the required memory and time. However, where computational resources are not an issue, the take home message is that the more trees the better.

As a randomized algorithm, it is finally interesting to study the standard deviation of the errors for Proximity Forest and how it varies with the number of trees. This is what we present in Fig. 9 where the y-axis represents the standard deviation on error-rate for 100 trees as a function of the standard deviation on k equals to 5, 10, and 50 trees. Each point represents a single dataset. One can see that the standard deviation reduces as we increase the number of trees, and that the magnitude of this improvement reduces when increasing k . Results for 50 trees are starting to be relatively close to the $y = x$ line, showing that only marginal improvements could be expected when going to $k > 100$.

4.3.2 Split selection using the Gini index

This section explores the influence of the number of candidates r that are randomly selected at each node. As a reminder, a set of r candidates—exemplars and parametrized distance measures—is evaluated at each node based on the Gini index. The one maximizing the Gini index is retained. To evaluate the influence of r , the

Fig. 8 Ratio of the error rates of Proximity Forest models: 100 trees over 10 trees (x-axis) against 100 trees over 50 trees (y-axis). A value of less than 1 on either axis indicates that the model with 100 trees has higher accuracy

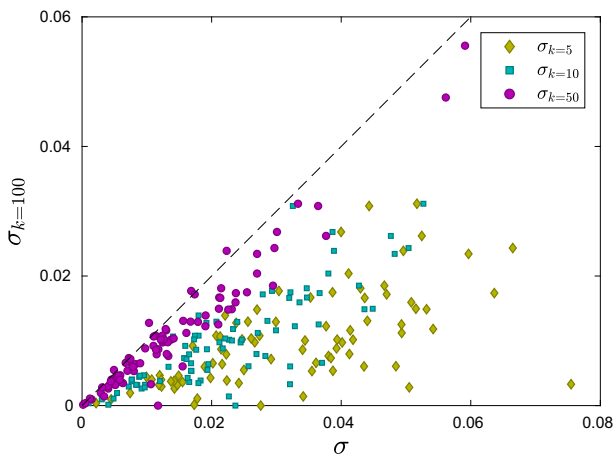
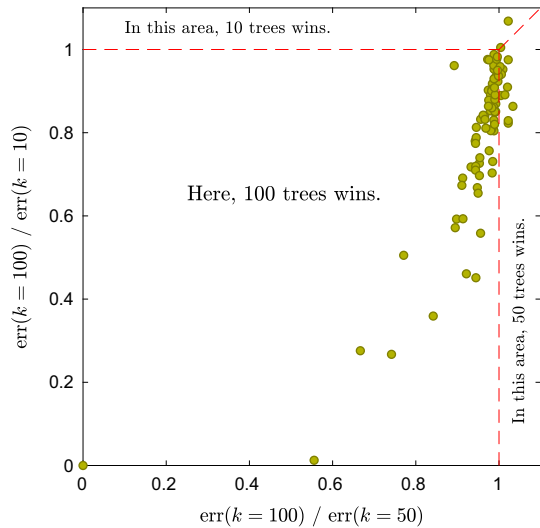
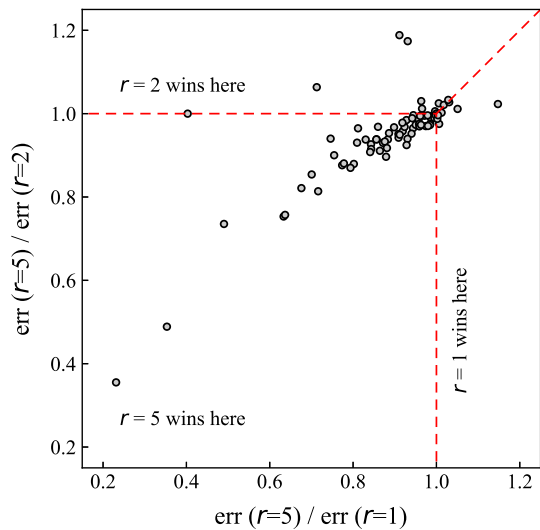


Fig. 9 Standard deviations σ of error rates on the 85 datasets of the UCR archive for Proximity Forest models: 100 trees against 50, 10 and 5 trees

UCR experiments were repeated for 1, 2, and 5 candidates on 100 trees. The results are averaged over 10 runs.

Figure 10 compares the classification accuracy for 5 candidates against 1 and 2 candidates. Each point represents the ratio of the error for 5 candidates to that for the alternative on a UCR dataset. Choosing between 5 candidates results in higher accuracy for most datasets. More precisely, selecting between 5 candidates results in greater accuracy than either 1 or 2 candidates on 61 datasets. Increasing the number of candidates leads to a reduction of the randomness on each node by discarding the worse splitters. Accordingly, the overall Proximity Forest accuracies are improved.

Fig. 10 Ratio of the error rates of Proximity Forest models: 5 candidates over 1 candidate (x-axis) against 5 candidates over 2 candidates (y-axis). A value of less than 1 on either axis suggests that the model with 5 candidates has superior accuracy



Increasing the number of candidates to more than 5 may further improve the classification accuracy. However, increasing the number of candidates per node has substantial impact on training time. Indeed, the analysis of the Proximity Forest's computational complexity in Sect. 3.4 shows that the training time scales linearly with the number of candidates. To verify this analysis, we compare both training and testing time of Proximity Forest for 1 and 5 candidates in Fig. 11. The testing time is displayed per query. Each point represents a dataset. The results show a mean increase of 4.6 times in training time between 1 and 5 candidates, and a mean decrease of 0.93 times in testing time.

It is notable that selection between multiple alternatives both reduces testing time and increases the training time by slightly less than the expected multiple of 5 times. This is because it results in slightly shallower trees. Selection of better splits better separates the classes, requiring fewer splits to obtain pure nodes that are made into leaves.

The tuning of the number of candidates is therefore driven by a trade-off between accuracy and time.

5 Conclusion

We introduced Proximity Forest: a novel, scalable algorithm for accurate time series classification. Motivated by a need for an algorithm that could learn from millions of time series, Proximity Forest is an ensemble of trees with a novel splitting criterion that makes it possible to make the most of decades of work in designing time series measures. In our case study, we demonstrated that Proximity Forest scales quasi-linearly with the quantity of training data, whereas most state of the art algorithms scale quadratically. Our experiments on the UCR datasets show that Proximity Forest

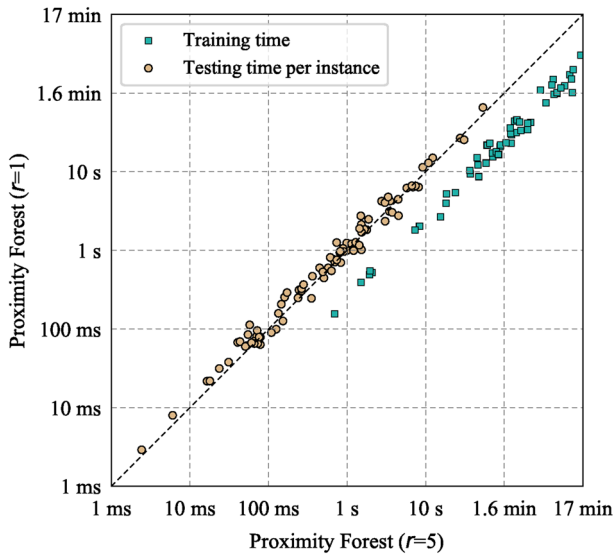


Fig. 11 Training and testing time of Proximity Forest for 1 and 5 candidates on UCR datasets

is not only very fast. It also has highly competitive accuracy relative to the current state-of-the-art, and is significantly more accurate than EE.

We believe that there are a number of improvements that can be explored to increase the accuracy of Proximity Forest while maintaining its quasi-linear complexity, such as improving the randomized selection of parameters for the distance measures—the current strategy was designed primarily to emulate EE as directly as possible. We would also like to investigate to what extent this novel algorithm might shed new light on the task of time series indexing.

Supplementary material

To ensure reproducibility, we make available the results of the experiments as well as our source code at <https://github.com/fpetitjean/ProximityForest/>.

Acknowledgements This research was supported by the Australian Research Council under Grant DE170100037. This material is based upon work supported by the Air Force Office of Scientific Research, Asian Office of Aerospace Research and Development (AOARD) under award number FA2386-17-1-4036. We are grateful to the editor and anonymous reviewers whose suggestions and comments have greatly strengthened the paper. The authors would also like to thank Prof Eamonn Keogh and all of the people who have contributed to the UCR time series classification archive.

Appendix A: Detailed UCR results

See Table 1.

Table 1 Detailed UCR results for five state-of-the-art algorithms and Proximity Forest

	<i>Train</i>	<i>Test</i>	<i>l</i>	<i>c</i>	DTW	BOSS	ST	EE	COTE	PF
Adiac	390	391	176	37	60.87	76.47	78.26	66.5	79.03	73.4
ArrHead	36	175	251	3	80.0	83.43	73.71	81.14	81.14	87.54
Beef	30	30	470	5	66.67	80.0	90.0	63.33	86.67	72.0
BeetFly	20	20	512	2	65.0	90.0	90.0	75.0	80.0	87.5
BirdChi	20	20	512	2	70.0	95.0	80.0	80.0	90.0	86.5
Car	60	60	577	4	76.67	83.33	91.67	83.33	90.0	84.67
CBF	30	900	128	3	99.44	99.78	97.44	99.78	99.56	99.33
ChloCon	467	3840	166	3	65.0	66.09	69.97	65.62	72.71	63.39
CinCECG	40	1380	1639	4	93.04	88.7	95.43	94.2	99.49	93.43
Coffee	28	28	286	2	100.0	100.0	96.43	100.0	100.0	100.0
Comput	250	250	720	2	62.4	75.6	73.6	70.8	74.0	64.44
CricketX	390	390	300	12	77.95	73.59	77.18	81.28	80.77	80.21
CricketY	390	390	300	12	75.64	75.38	77.95	80.51	82.56	79.38
CricketZ	390	390	300	12	73.59	74.62	78.72	78.21	81.54	80.1
DiaSizRed	16	306	345	4	93.46	93.14	92.48	94.44	92.81	96.57
DisPhAG	400	139	80	3	62.59	74.82	76.98	69.06	74.82	73.09
DisPhOC	600	276	80	2	72.46	72.83	77.54	72.83	76.09	79.28
DisPhTW	400	139	80	6	63.31	67.63	66.19	64.75	69.78	65.97
Earthqua	322	139	512	2	72.66	74.82	74.1	74.1	74.82	75.4
ECG200	100	100	96	2	88.0	87.0	83.0	88.0	88.0	90.9
ECG5000	500	4500	140	5	92.51	94.13	94.38	93.87	94.6	93.65
ECG5days	23	861	136	2	79.67	100.0	98.37	82.0	99.88	84.92
ElecDev	8926	7711	96	7	63.08	79.92	74.7	66.29	71.33	70.6
FaceAll	560	1690	131	14	80.77	78.17	77.87	84.85	91.78	89.38
FaceFour	24	88	350	4	89.77	100.0	85.23	90.91	89.77	97.39
FacesUCR	200	2050	131	14	90.78	95.71	90.59	94.49	94.24	94.59
50Words	450	455	270	50	76.48	70.55	70.55	81.98	79.78	83.14
Fish	175	175	463	7	83.43	98.86	98.86	96.57	98.29	93.49
FordA	3601	1320	500	2	66.52	92.95	97.12	73.79	95.68	85.46
FordB	3636	810	500	2	59.88	71.11	80.74	66.17	80.37	71.49
GunPoint	50	150	150	2	91.33	100.0	100.0	99.33	100.0	99.73
Ham	109	105	431	2	60.0	66.67	68.57	57.14	64.76	66.0
HandOut	1000	370	2709	2	87.84	90.27	93.24	88.92	91.89	92.14
Haptics	155	308	1092	5	41.56	46.1	52.27	39.29	52.27	44.45
Herring	64	64	512	2	53.12	54.69	67.19	57.81	62.5	57.97
InlSkate	100	550	1882	7	38.73	51.64	37.27	46.0	49.45	54.18
InsWinSou	220	1980	256	11	57.37	52.32	62.68	59.49	65.25	61.87
ItPowDem	67	1029	24	2	95.53	90.86	94.75	96.21	96.11	96.71
LaKitAp	375	375	720	3	79.47	76.53	85.87	81.07	84.53	78.19

Table 1 continued

	<i>Train</i>	<i>Test</i>	<i>l</i>	<i>c</i>	DTW	BOSS	ST	EE	COTE	PF
Light2	60	61	637	2	86.89	83.61	73.77	88.52	86.89	86.56
Light7	70	73	319	7	71.23	68.49	72.6	76.71	80.82	82.19
Mallat	55	2345	1024	8	91.43	93.82	96.42	93.99	95.39	95.76
Meat	60	60	448	3	93.33	90.0	85.0	93.33	91.67	93.33
MedImg	381	760	99	10	74.74	71.84	66.97	74.21	75.79	75.82
MidPhAG	400	154	80	3	51.95	54.55	64.29	55.84	63.64	56.23
MidPhOC	600	291	80	2	76.63	78.01	79.38	78.35	80.41	83.64
MidPhTW	399	154	80	6	50.65	54.55	51.95	51.3	57.14	52.92
MotStr	20	1252	84	2	86.58	87.86	89.7	88.26	93.69	90.24
NoECGT1	1800	1965	750	42	82.9	83.82	94.96	84.58	93.13	90.66
NoECGT2	1800	1965	750	42	87.02	90.08	95.11	91.35	94.55	93.99
OliveOil	30	30	570	4	86.67	86.67	90.0	86.67	90.0	86.67
OSULeaf	200	242	427	6	59.92	95.45	96.69	80.58	96.69	82.73
PhalOC	1800	858	80	2	76.11	77.16	76.34	77.27	77.04	82.35
Phoneme	214	1896	1024	39	22.68	26.48	32.07	30.49	34.92	32.01
Plane	105	105	144	7	100.0	100.0	100.0	100.0	100.0	100.0
ProPhAG	400	205	80	3	78.54	83.41	84.39	80.49	85.37	84.63
ProPhOC	600	291	80	2	79.04	84.88	88.32	80.76	86.94	87.32
ProPhTW	400	205	80	6	76.1	80.0	80.49	76.59	78.05	77.9
RefrigDev	375	375	720	3	44.0	49.87	58.13	43.73	54.67	53.23
ScrType	375	375	720	3	41.07	46.4	52.0	44.53	54.67	45.52
ShapSim	20	180	500	2	69.44	100.0	95.56	81.67	96.11	77.61
ShapAll	600	600	512	60	80.17	90.83	84.17	86.67	89.17	88.58
SmKitAp	375	375	720	3	67.2	72.53	79.2	69.6	77.6	74.43
SonyAIR1	20	601	70	2	69.55	63.23	84.36	70.38	84.53	84.58
SonyAIR2	27	953	65	2	85.94	85.94	93.39	87.83	95.17	89.63
StarCur	1000	8236	1024	3	89.83	97.78	97.85	92.61	97.96	98.13
Strawber	613	370	235	2	94.59	97.57	96.22	94.59	95.14	96.84
SwedLeaf	500	625	128	15	84.64	92.16	92.8	91.52	95.52	94.66
Symbols	25	995	398	6	93.77	96.68	88.24	95.98	96.38	96.16
SynCon	300	300	60	6	98.33	96.67	98.33	99.0	100.0	99.53
ToeSeg1	40	228	277	2	75.0	93.86	96.49	82.89	97.37	92.46
ToeSeg2	36	130	343	2	90.77	96.15	90.77	89.23	91.54	86.23
Trace	100	100	275	4	99.0	100.0	100.0	99.0	100.0	100.0
2LeECG	23	1139	82	2	86.83	98.07	99.74	97.1	99.3	98.86
2Patterns	1000	4000	128	4	99.85	99.3	95.5	100.0	100.0	99.96
UWaAll	1000	6164	152	8	96.23	93.89	94.22	96.85	96.43	97.23
UWaX	896	3582	315	8	77.44	76.21	80.29	80.54	82.19	82.86
UWaY	896	3582	315	8	70.18	68.51	73.03	72.56	75.85	76.15

Table 1 continued

	<i>Train</i>	<i>Test</i>	<i>l</i>	<i>c</i>	DTW	BOSS	ST	EE	COTE	PF
UWaZ	896	3582	945	8	67.5	69.49	74.85	72.36	75.04	76.4
Wafer	896	3582	315	2	99.59	99.48	100.0	99.74	99.98	99.55
Wine	57	54	234	2	61.11	74.07	79.63	57.41	64.81	56.85
WordSyn	267	638	270	25	74.92	63.79	57.05	77.9	75.71	77.87
Worms	181	77	900	5	53.25	55.84	74.03	66.23	62.34	71.82
Worms2	181	77	900	2	58.44	83.12	83.12	68.83	80.52	78.44
Yoga	300	3000	426	2	84.3	91.83	81.77	87.9	87.67	87.86
Av. rank					5.18	3.65	3.08	3.95	2.28	2.88
Wins					3	20	30	8	26	22

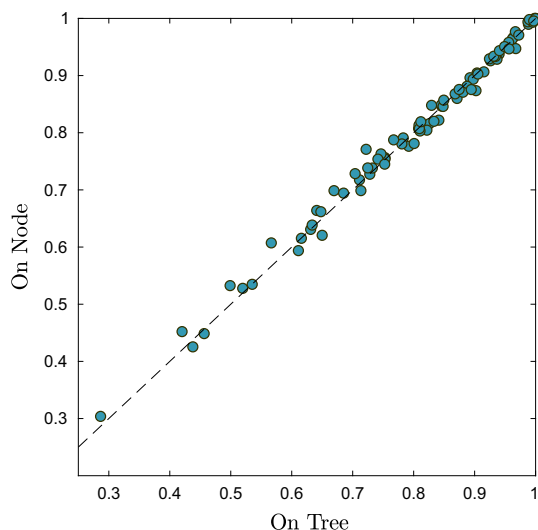
Bold values indicate the best accuracy scores. Proximity Forest results are obtained for 100 trees and 5 candidates, and averaged over 10 runs

Appendix B: On a variation of the proximity forest

We decided to explore another variant of the Proximity Forest algorithm by randomly selecting a distance measure for each tree, rather than for each node. In this new variant, only the exemplars and the parameters of the distance-metric are randomly chosen at each node. The UCR experiments were repeated for 100 trees and 1 candidate for this new ‘on tree’ variant. Each Proximity Forest result is averaged over 50 runs.

Figure 12 compares classification accuracy for the original version ‘on node’, presented in Sect. 3.2, and the proposed variant ‘on tree’. Each point represents a single dataset of the UCR dataset. The number of trees has been fixed to 100.

Fig. 12 Accuracy of Proximity Forest when randomly selecting the distance measure ‘on node’ and ‘on tree’



The results show a slight advantage for the ‘on node’ approach with 44 wins, 39 losses and 2 ties. Where the ‘on tree’ variant uses a single distance measure per tree, the ‘on node’ variant allows multiple combinations of measures in a single tree, thus making it more robust to inefficient metrics.

References

- Bagnall A, Lines J, Bostrom A, Large J, Keogh E (2017) The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Min Knowl Discov* 31(3):606–660
- Bagnall A, Lines J, Hills J, Bostrom A (2015) Time-series classification with COTE: the collective of transformation-based ensembles. *IEEE Trans Knowl Data Eng* 27(9):2522–2535
- Balakrishnan S, Madigan D (2006) Decision trees for functional variables. In: *IEEE international conference on data mining (ICDM-06)*, pp 798–802
- Bernhardsson E (2013) Indexing with annoy. <https://github.com/spotify/annoy>. Accessed 23 March 2018
- Breiman L (2001) Random forests. *Mach Learn* 45(1):5–32
- Chen L, Ng R (2004) On the marriage of lp-norms and edit distance. In: *Proceedings of the thirtieth international conference on very large data bases*, vol 30, pp 792–803. VLDB Endowment
- Chen L, Özsu M T, Oria V (2005) Robust and fast similarity search for moving object trajectories. In: *Proceedings of the 2005 ACM SIGMOD international conference on management of data*, pp 491–502. ACM
- Chen Y, Keogh E, Hu B, Begum N, Bagnall A, Mueen A, Batista G (2015) The UCR time series classification archive. www.cs.ucr.edu/~eamonn/time_series_data/. Accessed 23 March 2018
- Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. *J Mach Learn Res* 7:1–30
- Deng H, Runger G, Tuv E, Vladimir M (2013) A time series forest for classification and feature extraction. *Inf Sci* 239:142–153
- Douzal-Chouakria A, Amblard C (2012) Classification trees for time series. *Pattern Recognit* 45(3):1076–1091
- Geurts P, Ernst D, Wehenkel L (2006) Extremely randomized trees. *Mach Learn* 63(1):3–42
- Górecki T, Łuczak M (2013) Using derivatives in time series classification. *Data Min Knowl Discov* 26(2):310–331
- Grabocka J, Wistuba M, Schmidt-Thieme L (2016) Fast classification of univariate and multivariate time series through shapelet discovery. *Knowl Inf Syst* 49(2):429–454
- Haghiri S, Ghoshdastidar D, von Luxburg U (2017) Comparison-based nearest neighbor search. *arXiv e-prints*, [arXiv:1704.01460](https://arxiv.org/abs/1704.01460)
- Haghiri S, Garreau D, von Luxburg U (2018) Comparison-based random forests. *arXiv e-prints*, [arXiv:1806.06616](https://arxiv.org/abs/1806.06616)
- Hamooni H, Mueen A (2014) Dual-domain hierarchical classification of phonetic time series. In: *2014 IEEE international conference on data mining*, pp 160–169. IEEE
- Hills J, Lines J, Baranauskas E, Mapp J, Bagnall A (2014) Classification of time series by shapelet transformation. *Data Min Knowl Discov* 28(4):851–881
- Ho TK (1995) Random decision forests. In: *Proceedings of the third international conference on document analysis and recognition*, 1995, vol 1, pp 278–282. IEEE
- Jeong YS, Jeong MK, Omitaomu OA (2011) Weighted dynamic time warping for time series classification. *Pattern Recognit* 44(9):2231–2240
- Karlsson I, Papapetrou P, Boström H (2016) Generalized random shapelet forests. *Data Min Knowl Discov* 30(5):1053–1085
- Keogh E, Wei L, Xi X, Lee S H, Vlachos M (2006) LB_Keogh supports exact indexing of shapes under rotation invariance with arbitrary representations and distance measures. In: *Proceedings of the 32nd international conference on very large data bases*, pp 882–893. VLDB Endowment
- Keogh EJ, Pazzani MJ (2001) Derivative dynamic time warping. In: *Proceedings of the 2001 SIAM international conference on data mining*, pp 1–11. SIAM
- Lemire D (2009) Faster retrieval with a two-pass dynamic-time-warping lower bound. *Pattern Recognit* 42(9):2169–2180
- Lifshits Y (2010) Nearest neighbor search: algorithmic perspective. *SIGSPATIAL Spec* 2(2):12–15

- Lin J, Khade R, Li Y (2012) Rotation-invariant similarity in time series using bag-of-patterns representation. *J Intell Inf Syst* 39(2):287–315
- Lines J, Bagnall A (2015) Time series classification with ensembles of elastic distance measures. *Data Min Knowl Discov* 29(3):565
- Marteau PF (2009) Time warp edit distance with stiffness adjustment for time series matching. *IEEE Trans Pattern Anal Mach Intell* 31(2):306–318
- Marteau PF (2016) Times series averaging and denoising from a probabilistic perspective on time-elastic kernels. *arXiv preprint*, [arXiv:1611.09194](https://arxiv.org/abs/1611.09194)
- Muja M. FLANN-Fast library for approximate nearest neighbors. www.cs.ubc.ca/research/flann/. Accessed 23 March 2018
- Pekalska E, Duin RP, Paclík P (2006) Prototype selection for dissimilarity-based classifiers. *Pattern Recognit* 39(2):189–208
- Petitjean F, Forestier G, Webb GI, Nicholson AE, Chen Y, Keogh E (2014) Dynamic time warping averaging of time series allows faster and more accurate classification. In: 2014 IEEE international conference on data mining, pp 470–479. IEEE
- Petitjean F, Forestier G, Webb GI, Nicholson AE, Chen Y, Keogh E (2016) Faster and more accurate classification of time series by exploiting a novel dynamic time warping averaging algorithm. *Knowl Inf Syst* 47(1):1–26
- Petitjean F, Gançarski P (2012) Summarizing a set of time series by averaging: from Steiner sequence to compact multiple alignment. *Theor Comput Sci* 414(1):76–91
- Rakthanmanon T, Keogh E (2013) Fast shapelets: a scalable algorithm for discovering time series shapelets. In: Proceedings of the 13th SIAM international conference on data mining, pp 668–676. SIAM
- Sakoe H, Chiba S (1971) A dynamic programming approach to continuous speech recognition. In: Proceedings of the seventh international congress on acoustics, vol 3, pp 65–69. Budapest, Hungary
- Sakoe H, Chiba S (1978) Dynamic programming algorithm optimization for spoken word recognition. *IEEE Trans Acoust Speech Signal Process* 26(1):43–49
- Sathe S, Aggarwal CC (2017) Similarity forests. In: Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining, KDD '17, pp 395–403. ACM. <https://doi.org/10.1145/3097983.3098046>
- Schäfer P (2015) The BOSS is concerned with time series classification in the presence of noise. *Data Min Knowl Discov* 29(6):1505–1530
- Schäfer P (2015) Scalable time series classification. *Data Min Knowl Discov* 2:1–26
- Schäfer P, Höggqvist M (2012) SFA: a symbolic fourier approximation and index for similarity search in high dimensional datasets. In: Proceedings of the 15th international conference on extending database technology, EDBT '12, pp 516–527. ACM. <https://doi.org/10.1145/2247596.2247656>
- Schäfer P, Leser U (2017) Fast and accurate time series classification with WEASEL. In: Proceedings of the 2017 ACM conference on information and knowledge management, pp 637–646. ACM
- Senin P, Malinchik S (2013) SAX-VSM: Interpretable time series classification using SAX and vector space model. In: 2013 IEEE 13th international conference on data mining, pp 1175–1180. IEEE
- Stefan A, Athitsos V, Das G (2013) The move-split-merge metric for time series. *IEEE Trans Knowl Data Eng* 25(6):1425–1438
- Tan CW, Webb GI, Petitjean F (2017) Indexing and classifying gigabytes of time series under time warping. In: Proceedings of the 2017 SIAM international conference on data mining, pp 282–290. SIAM
- Ting K.M, Zhu Y, Carman M, Zhu Y, Zhou Z.H (2016) Overcoming key weaknesses of distance-based neighbourhood methods using a data dependent dissimilarity measure. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 1205–1214. ACM
- Ueno K, Xi X, Keogh E, Lee DJ (2006) Anytime classification using the nearest neighbor algorithm with applications to stream mining. In: 6th international conference on data mining, 2006. ICDM'06, pp 623–632. IEEE
- Vlachos M, Hadjieleftheriou M, Gunopulos D, Keogh E (2006) Indexing multidimensional time-series. *Int J Very Large Data Bases* 15(1):1–20
- Wang X, Mueen A, Ding H, Trajcevski G, Scheuermann P, Keogh E (2013) Experimental comparison of representation methods and distance measures for time series data. *Data Min Knowl Discov* 26(2):275–309
- Yamada Y, Suzuki E, Yokoi H, Takabayashi K (2003) Decision-tree induction from time-series data based on a standard-example split test. In: Proceedings of the twentieth international conference on international

conference on machine learning, ICML'03, pp 840–847. AAAI Press. <http://dl.acm.org/citation.cfm?id=3041838.3041944>

Ye L, Keogh E (2011) Time series shapelets: a novel technique that allows accurate, interpretable and fast classification. *Data Min Knowl Discov* 22(1):149–182

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Affiliations

Benjamin Lucas¹  · Ahmed Shifaz¹  · Charlotte Pelletier¹  ·
Lachlan O'Neill¹ · Nayyar Zaidi¹ · Bart Goethals¹  · François Petitjean¹  ·
Geoffrey I. Webb¹ 

Ahmed Shifaz
ahmed.shifaz@monash.edu

Charlotte Pelletier
charlotte.pelletier@monash.edu

Lachlan O'Neill
lsone1@monash.edu

Nayyar Zaidi
nayaar.zaidi@monash.edu

Bart Goethals
bart.goethals@monash.edu

François Petitjean
francois.petitjean@monash.edu

Geoffrey I. Webb
geoff.webb@monash.edu

¹ Faculty of Information Technology, Monash University, 25 Exhibition Walk, Melbourne, VIC 3800, Australia